PRUEBA DE CONOCIMIENTOS

Desarrollador TI / Sénior

Plataforma de Gestión de Pólizas de Arrendamiento

Candidato	**Brayan Julián Barrantes Medina**
Fecha	**16 de octubre de 2026**
Repositorio (Módulo 2)	**github.com/JulianMediina/prueba-polizas**
Demo en línea	**d3l78drj5ly5ij.cloudfront.net**

Contenido

Módulo 1 – Diseño de sistema: Plataforma de Gestión de Pólizas

1. Contexto y supuestos
2. Arquitectura de alto nivel (AWS)
3. Tres patrones de arquitectura
4. Modelo de datos principal
5. Flujos principales
6. Atributos de calidad
7. Seguridad (transversal)
8. Decisiones registradas

Módulo 2 – Prueba técnica práctica

Módulo 3 – Optimización de la consulta de pedidos por país

Diagnóstico: por qué es lenta
Estrategias (priorizadas)
Resumen
Scripts

Módulo 4 – Incorporar un fix de seguridad de main en feature/new-login

Respuesta corta
Paso a paso
Casos especiales
¿Qué pasa cuando la feature se integre a main?
Alternativas descartadas
Buenas prácticas de proceso
Demostración reproducible

Módulo 5 – Liderazgo técnico y gestión

1. Cinco prioridades para las primeras 2 semanas
2. Organización del equipo para mejorar velocidad y calidad
3. Métricas para evaluar el desempeño del área
4. Prácticas técnicas obligatorias
5. Gestionar la presión del negocio sin comprometer la calidad

Anexo – Registros de decisiones de arquitectura (ADR)

ADR 0001 – Microservicios con API Gateway
ADR 0002 – Event-driven con Transactional Outbox
ADR 0003 – Arquitectura hexagonal y capa anticorrupción para el CORE
ADR 0004 – Despliegue demo en una EC2 con CloudFront

Módulo 1 – Diseño de sistema: Plataforma de Gestión de Pólizas

> Arquitectura objetivo de producción en AWS. La implementación del Módulo 2 y el despliegue demo son una versión reducida y consciente de este diseño (ver ADR 0004).

1. Contexto y supuestos

Actores

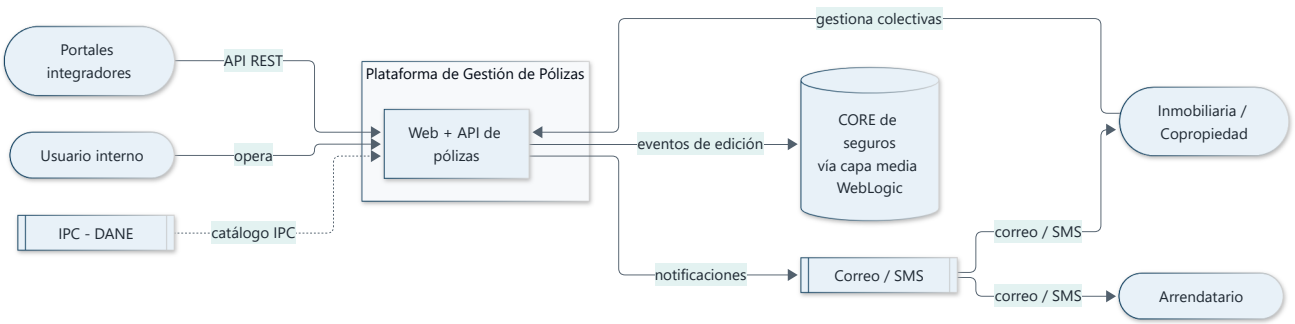
Actor	Rol en la póliza
Inmobiliaria / administración de copropiedad	Tomador de la póliza **colectiva**
Arrendatario	Asegurado (en la individual también es el tomador)
Arrendador	Beneficiario en caso de siniestro
Usuario interno de la aseguradora	Opera la plataforma (back-office)
Integradores (portales de inmobiliarias)	Consumen la API con credenciales de máquina
CORE de seguros (legado)	Sistema de registro; se actualiza a través de un servicio agnóstico de edición en WebLogic

Supuestos explícitos

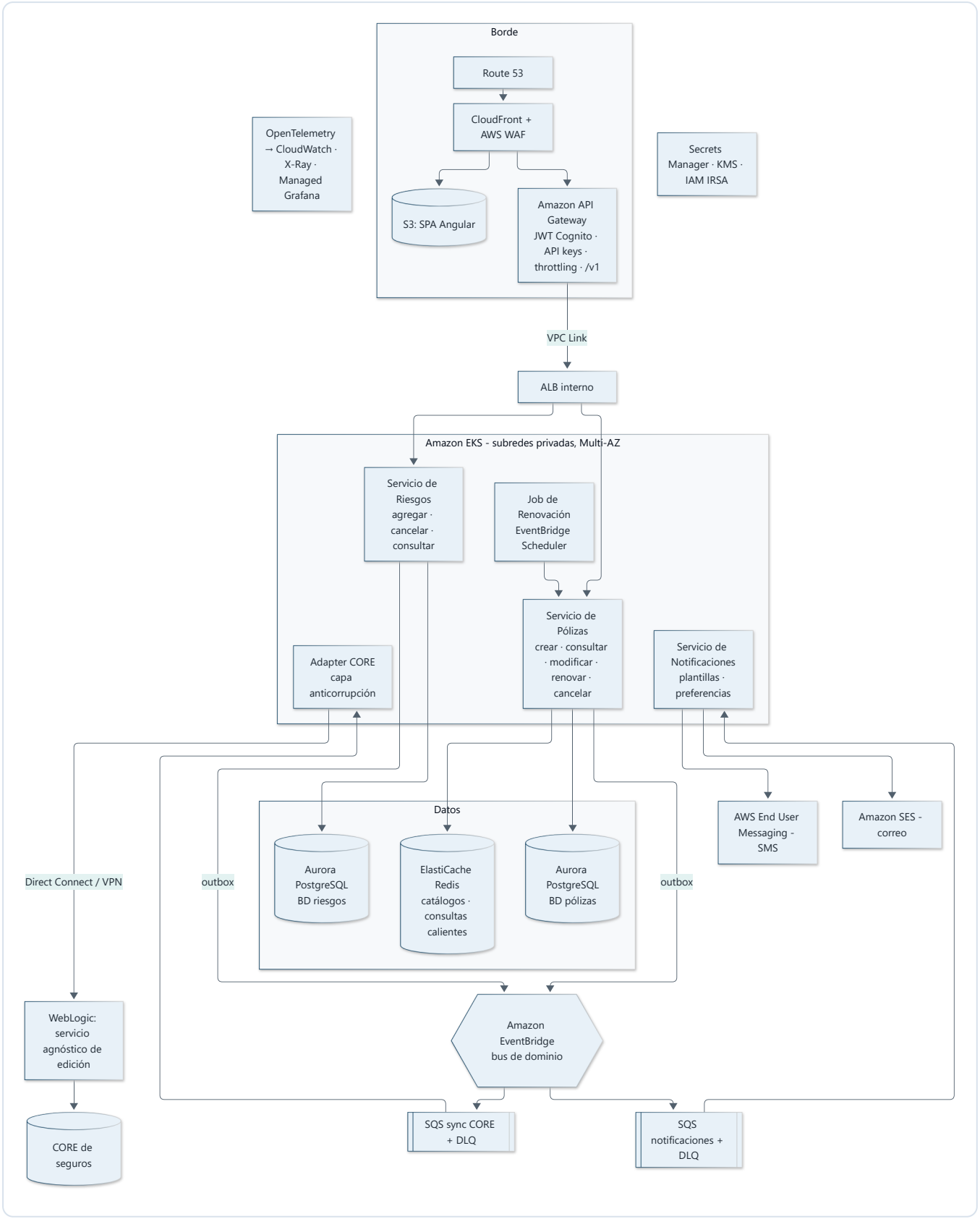
1. El **canon de la póliza es la suma de los cánones de sus riesgos activos**. En la individual coincide con el canon de su único riesgo; en la colectiva cada inmueble tiene su canon. **Prima = canon × meses de vigencia.**
2. La **renovación** se hace por el mismo periodo inicial, empieza donde termina la vigencia actual y ajusta el canon de cada riesgo con el **IPC del año anterior** al inicio de la nueva vigencia (fuente DANE). Se actualiza la misma póliza (estado `RENOVADA`) y se guarda el historial en `renovacion`.
3. El riesgo único de una póliza individual no se cancela por separado: se cancela la póliza. Una colectiva conserva al menos un riesgo activo.
4. El CORE es la fuente de verdad contable; la plataforma es la fuente de verdad operativa y le informa cada cambio de estado. El CORE puede estar lento o caído sin que la plataforma deje de operar.

2. Arquitectura de alto nivel (AWS)

2.1 C4 – Nivel 1: contexto



2.2 C4 – Nivel 2: contenedores



2.3 Componentes

Componente	Responsabilidad	Tecnología
API Gateway	Punto único de entrada: autenticación (JWT de Cognito para personas, API keys + planes de uso para integradores), *throttling*, versionado por ruta, validación básica	Amazon API Gateway (REST) + WAF
Servicio de Pólizas	Ciclo de vida de la póliza; reglas de vigencia, prima y renovación; dueño del agregado `Poliza`	Spring Boot en EKS
Servicio de Riesgos	Riesgos de las colectivas (agregar, cancelar) y el riesgo único de la individual; consultas por inmueble/arrendatario	Spring Boot en EKS
Job de Renovación	Selecciona pólizas por vencer (ventana configurable) y las renueva por lotes idempotentes	Spring Batch disparado por EventBridge Scheduler
Servicio de Notificaciones	Consume eventos `PolizaCreada` / `PolizaRenovada`, resuelve plantilla y canal, envía correo y SMS	SES + End User Messaging, plantillas versionadas
Adapter CORE	Traduce eventos de dominio al contrato del servicio de edición de WebLogic; aísla protocolo (SOAP/REST), reintentos y errores	Spring Boot + Resilience4j
Base de datos	Una base lógica por servicio; Multi-AZ; réplicas de lectura	Aurora PostgreSQL
Bus de eventos	Desacopla productores de consumidores; reglas por tipo de evento; archivo y *replay*	EventBridge → SQS (+ DLQ)

> Nota sobre el dominio: pólizas y riesgos comparten invariantes (una individual tiene 1 riesgo; el canon de la póliza depende de sus riesgos). En la práctica se recomienda **empezar con un único servicio "Pólizas" que contenga ambos módulos** y separar Riesgos solo cuando su carga lo justifique. Si se separan, la invariante se protege con validación síncrona (Riesgos consulta a Pólizas) y el recálculo de canon/prima por evento.

3. Tres patrones de arquitectura

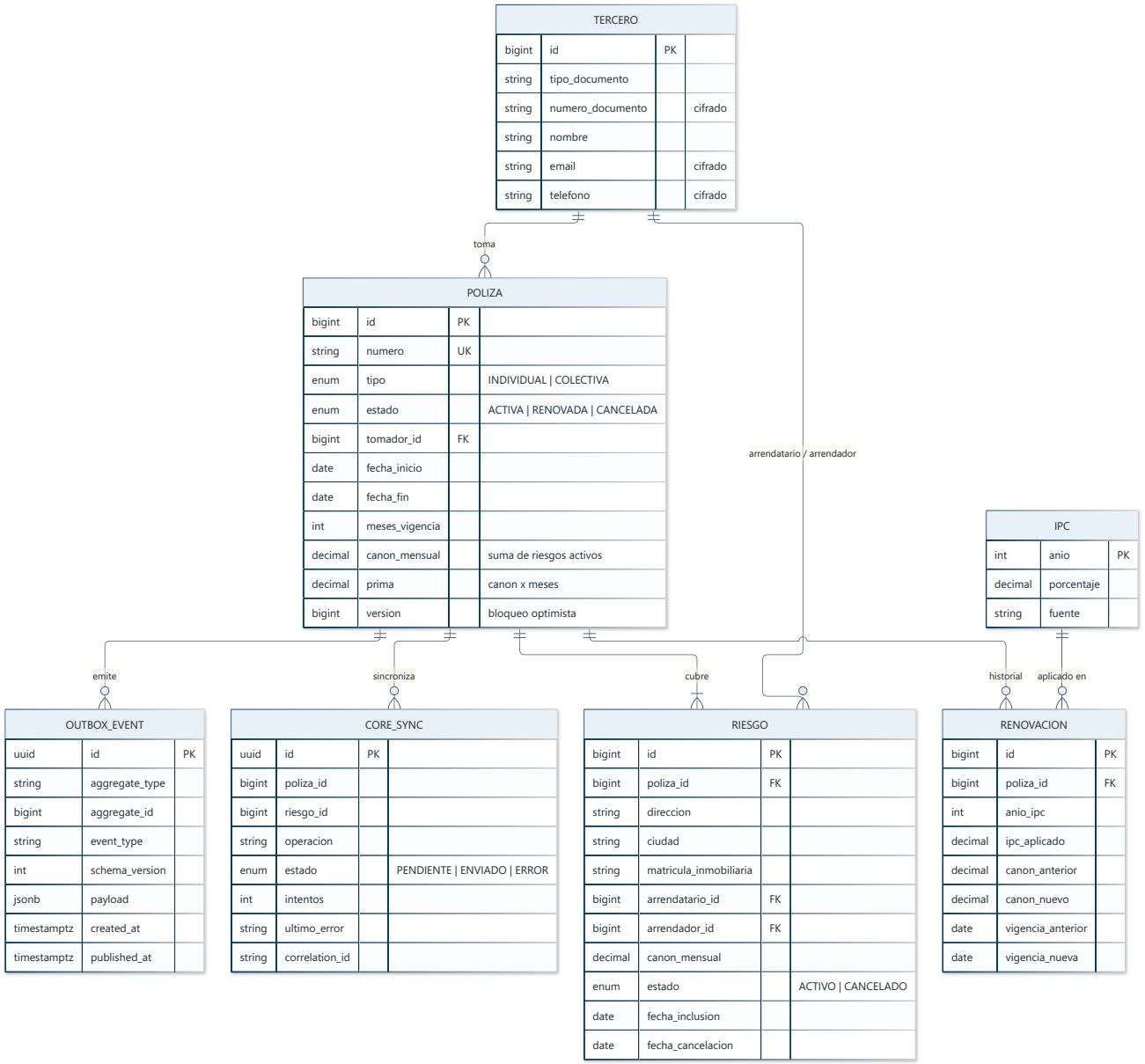
#	Patrón	Por qué en este caso	Costo / trade-off
1	**API Gateway + microservicios**	Cargas y ritmos distintos: consultas intensivas del back-office, renovación por lotes nocturna y notificaciones en ráfagas. Desplegar y escalar por separado evita que una renovación masiva degrade la consulta. El gateway centraliza autenticación, *rate limiting* por integrador y versionado.	Más piezas que operar; mitigado con plantillas de servicio, observabilidad común y un único repositorio de IaC.
2	**Event-Driven con Transactional Outbox**	Notificar y sincronizar con el CORE **no debe bloquear ni tumbar** la transacción de negocio. El evento se escribe en la tabla `outbox_event` en la misma transacción que el cambio; un *relay* lo publica en EventBridge. Garantiza "al menos una vez" sin transacciones distribuidas.	Consistencia eventual con el CORE; consumidores idempotentes (clave `eventId`).

#	Patrón	Por qué en este caso	Costo / trade-off
3	**Hexagonal + capa anticorrupción (ACL) hacia el CORE**	El CORE es legado y su contrato (SOAP/XML en WebLogic) no debe contaminar el dominio. El puerto `CorePolizasPort` define *qué* se informa; el adapter decide *cómo*. Permite probar sin el CORE (mock), cambiar de protocolo y concentrar resiliencia y traducción de errores en un solo lugar.	Una capa de mapeo adicional que mantener.

Complementarios: CQRS ligero para consultas (réplicas de lectura + caché), Saga orquestada para la renovación (renovar → CORE → notificar, con compensación si el CORE rechaza), Circuit Breaker / Retry / Bulkhead.

Cómo se ve en el código del Módulo 2: el dominio rico (`Poliza` como agregado raíz), el puerto `CorePolizasPort` con `CoreHttpAdapter` (timeout + retry + circuit breaker) y el envío al CORE **después del commit** y de forma asíncrona (`@TransactionalEventListener(AFTER_COMMIT)` + `@Async`). El outbox queda documentado como siguiente paso.

4. Modelo de datos principal

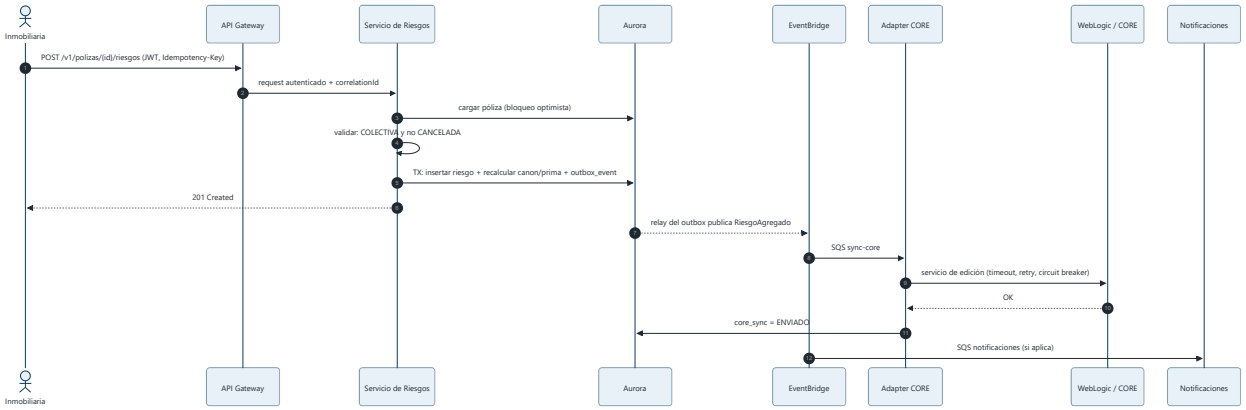


Invariantes: una INDIVIDUAL tiene exactamente un riesgo; una CANCELADA no se renueva ni recibe riesgos; cancelar la póliza cancela sus riesgos en la misma transacción; canon y prima siempre se derivan de los riesgos activos. Los datos personales (Ley 1581 de 2012) se cifran en columna o se tokenizan y nunca se escriben en logs.

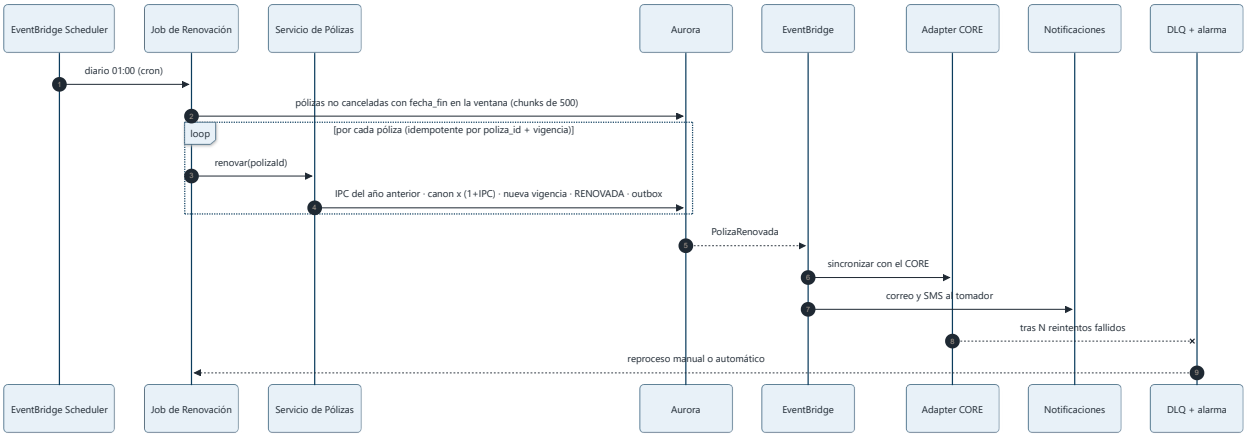
> En el código del Módulo 2 el tercero se simplifica a columnas embebidas (`tomador_*` , `arrendatario_*` , `arrendador_*`) para mantener el alcance; el modelo de arriba es el objetivo.

5. Flujos principales

5.1 Agregar un riesgo a una póliza colectiva



5.2 Renovación automática



Si falta el IPC del año requerido, la póliza no se renueva y se emite una alerta operativa (no se usa un valor supuesto).

6. Atributos de calidad

Escalabilidad

- Servicios sin estado en EKS con **HPA** (CPU/RPS) y **KEDA** según la profundidad de las colas SQS.
- Aurora con **réplicas de lectura** para consultas y reportes; Redis para catálogos (IPC) y consultas calientes.
- Renovación por **lotes particionados** por fecha de vencimiento, *chunks* idempotentes y concurrencia limitada para no saturar el CORE (bulkhead).
- **Paginación obligatoria** en listados e índices alineados a los filtros (`tipo, estado` , `fecha_fin`).

Logs y observabilidad

- Logs **JSON estructurados** (en el código: `logging.structured.format.console=ecs`) con `correlationId` y `traceId` propagados (W3C Trace Context) desde el gateway hasta el CORE.
- **OpenTelemetry** → CloudWatch Logs/Metrics + X-Ray (o Managed Prometheus + Grafana).
- Métricas **RED/USE** y de negocio: pólizas creadas/renovadas, fallos con el CORE, edad de mensajes en DLQ, latencia de notificaciones.
- **SLO** con alarmas: 99,9 % de disponibilidad, p95 < 300 ms en consultas, 0 mensajes en DLQ > 15 min.
- Sin datos personales en logs (enmascarado de documentos, correos y teléfonos).

Tolerancia a fallos

- **Multi-AZ** en EKS, Aurora y Redis; *PodDisruptionBudgets*, *readiness/liveness probes*.
- **Resilience4j** en el adapter: timeout, retry con backoff exponencial y *jitter*, circuit breaker, bulkhead.
- **Outbox + DLQ + reproceso**; consumidores **idempotentes**; `Idempotency-Key` en los POST.
- **Degradación controlada:** si el CORE no responde, la operación se confirma y queda `PENDIENTE` en `core_sync`; la reconciliación diaria compara estados con el CORE.
- Despliegues **blue/green o canary** con rollback automático por métricas.

Versionamiento de APIs

- Versión mayor en la URI (`/v1/...`) publicada en el gateway; cambios **compatibles** (campos nuevos opcionales) no suben versión.
- Cambios incompatibles → `/v2`, conviviendo con `/v1`, que se marca con los headers `Deprecation` y `Sunset` y una ventana de migración comunicada a integradores.
- Contratos **OpenAPI versionados** en el repositorio, *contract testing* (consumidor-proveedor) en CI.
- Eventos con `schemaVersion` y evolución **solo aditiva**; un cambio incompatible crea un tipo de evento nuevo.

7. Seguridad (transversal)

- Cognito (OIDC) para usuarios, API keys + planes de uso para integradores; autorización por rol/inmobiliaria.
- WAF (OWASP Top 10, *rate-based rules*), TLS 1.2+ extremo a extremo.
- Secrets Manager con rotación, KMS para cifrado en reposo, **IRSA** para que cada pod tenga solo sus permisos.
- Subredes privadas; conectividad privada con el datacenter de WebLogic (Direct Connect o Site-to-Site VPN).
- Escaneo de secretos, dependencias e imágenes en el pipeline (como en este repositorio: Gitleaks, Dependabot, Trivy, ECR scan on push).

8. Decisiones registradas

- ADR 0001 – Microservicios con API Gateway
- ADR 0002 – Event-driven con Transactional Outbox
- ADR 0003 – Hexagonal y capa anticorrupción para el CORE

- ADR 0004 – Despliegue demo en una EC2 con CloudFront

Módulo 2 – Prueba técnica práctica

Repositorio: https://github.com/JulianMediina/prueba-polizas

Demo en línea: https://d3l78drj5ly5ij.cloudfront.net (disponible durante el periodo de evaluación)

El código, las instrucciones de ejecución y los ejemplos `curl` de todos los endpoints están en el README del backend. Resumen de lo entregado:

Pedido del enunciado	Dónde está
Estructura Spring Boot por capas (controller, service, repository)	`backend/src/main/java/com/aseguradora/polizas`
Entidades básicas `Poliza` y `Riesgo`	`domain/Poliza.java`, `domain/Riesgo.java`
6 endpoints	`PolizaController`, `RiesgoController`
Validaciones de negocio	En el dominio (`Poliza`), cubiertas por tests unitarios y de integración
Mock `POST /core-mock/evento`	`CoreMockController`, invocado después de cada cambio de estado
Header `x-api-key`	`ApiKeyAuthFilter` (401 sin key o con key inválida)
README con instrucciones	`backend/README.md` y `README.md`

Adicional: interfaz web en Angular, 64 tests en el backend con cobertura ≥ 80 %, CI en GitHub Actions (tests, e2e con Docker Compose y escaneo de seguridad) y despliegue en AWS definido con Terraform.

Ejecución local: `cp .env.example .env` (con `API_KEY=123456`) y `docker compose up --build`. Frontend en `http://localhost:8080`; API directa en `http://localhost:8081` con el header `x-api-key: 123456`.

Módulo 3 – Optimización de la consulta de pedidos por país

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM   orders o
JOIN   customers c ON o.customer_id = c.customer_id
WHERE  c.country = 'México';
```

`orders`: 10 M filas · `customers`: 500 K filas · consulta frecuente y lenta.

Diagnóstico: por qué es lenta

Sin índices adecuados, el plan típico es **full scan de `customers`** (500 K filas para quedarse con las de México) y **full scan de `orders`** (10 M filas) con un *hash join*. El costo lo domina leer `orders` completa. Las causas más comunes:

1. No hay índice sobre la FK `orders.customer_id` (Oracle **no** lo crea automáticamente).
2. No hay índice sobre `customers.country`, o las estadísticas no reflejan la distribución sesgada del país.
3. Se leen columnas que obligan a ir a la tabla (`customer_name`, `order_date`, `total_amount`).
4. El texto `'México'` es frágil: tildes, mayúsculas y *collation* ("México" ≠ "Mexico").

Primer paso siempre: medir. En Oracle:

```
EXPLAIN PLAN FOR <consulta>;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
-- Plan real con estadísticas de ejecución:
SELECT /*+ GATHER_PLAN_STATISTICS */ ... ;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY_CURSOR(NULL, NULL, 'ALLSTATS LAST'));
```

En PostgreSQL: `EXPLAIN (ANALYZE, BUFFERS)`. Se compara *estimated rows* contra *actual rows*: si difieren mucho, el problema son las estadísticas antes que los índices.

Estrategias (priorizadas)

1. Estadísticas e histogramas al día

```
BEGIN
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'CUSTOMERS', method_opt => 'FOR COLUMNS SIZE AUTO COUNTRY', cascade => TRUE);
  DBMS_STATS.GATHER_TABLE_STATS(USER, 'ORDERS', cascade => TRUE);
END;
/
```

- **Qué hace:** el optimizador conoce cuántos clientes hay por país (histograma) y elige el plan correcto: si México es el 2 % de los clientes, conviene índice + *nested loops*; si es el 60 %, *full scan* + *hash join*.
- **Costo:** casi nulo. Es lo primero que se revisa.

2. Índices adecuados (la mejora más grande)

```
-- Filtro + columnas que se leen de customers: el filtro se resuelve solo con el índice
CREATE INDEX ix_customers_country_cov ON customers (country, customer_id, customer_name);

-- FK de orders + columnas que se proyectan: index-only scan, sin tocar la tabla de 10 M filas
CREATE INDEX ix_orders_customer_cov ON orders (customer_id, order_date, total_amount, order_id);
```

- **Qué hace:** el plan pasa a *index range scan* de los clientes de México → *nested loops* (o *hash join* si son muchos) contra el índice de `orders` por `customer_id`. Se leen solo los bloques de los pedidos de esos clientes.
- **Por qué mejora:** de leer 10 M filas a leer solo las de México (p. ej. 200 K), y de bloques de tabla a bloques de índice, que son más compactos.
- **Costo:** espacio en disco y más trabajo en cada `INSERT/UPDATE` de `orders`. Un índice de cobertura en la tabla más escrita debe justificarse; si no, basta `orders(customer_id)`.

3. Vista materializada (consulta frecuente, datos que toleran segundos/minutos de retraso)

```
CREATE MATERIALIZED VIEW LOG ON orders    WITH ROWID, SEQUENCE (customer_id, order_date, total_amount) INCLUDING NEW
VALUES;
CREATE MATERIALIZED VIEW LOG ON customers WITH ROWID, SEQUENCE (customer_id, customer_name, country)  INCLUDING NEW
VALUES;

CREATE MATERIALIZED VIEW mv_pedidos_por_pais
  BUILD IMMEDIATE
  REFRESH FAST ON COMMIT
  ENABLE QUERY REWRITE
AS
SELECT c.country, o.order_id, o.order_date, c.customer_name, o.total_amount,
       o.ROWID AS o_rid, c.ROWID AS c_rid
FROM   orders o JOIN customers c ON o.customer_id = c.customer_id;

CREATE INDEX ix_mv_pedidos_pais ON mv_pedidos_por_pais (country);
```

- **Qué hace:** precalcula el join; la consulta lee solo la porción de México. Con *query rewrite* la aplicación ni siquiera cambia su SQL.
- **Costo:** espacio y costo de refresco. `ON COMMIT` encarece cada transacción sobre `orders`; si se tolera retraso, mejor `REFRESH FAST ON DEMAND` programado (cada 5 minutos).

4. Particionamiento

```
-- orders por rango de fecha (intervalo mensual) + índice local
CREATE TABLE orders (
  order_id NUMBER PRIMARY KEY, customer_id NUMBER NOT NULL, order_date DATE NOT NULL, total_amount NUMBER(14,2)
)
PARTITION BY RANGE (order_date) INTERVAL (NUMTOYMINTERVAL(1, 'MONTH'))
(PARTITION p_inicial VALUES LESS THAN (DATE '2020-01-01'));

-- customers por lista de país
CREATE TABLE customers (...) PARTITION BY LIST (country) (
  PARTITION p_mx VALUES ('MX'), PARTITION p_co VALUES ('CO'), PARTITION p_otros VALUES (DEFAULT));
```

- **Qué hace:** *partition pruning*: si la consulta filtra por periodo ("pedidos de México del último mes"), Oracle lee solo esas particiones. Con *reference partitioning* `orders` puede heredar la partición de su cliente.
- **Costo:** licencia (Partitioning es una opción de Enterprise Edition), diseño de claves y mantenimiento. Aporta más cuando la consulta incluye un rango de fechas, que es lo usual en la práctica.

5. Reescritura de la consulta y del modelo

- Normalizar el país a un **código ISO** (`country_code = 'MX'`) con *check constraint*: evita problemas de tildes y *collation* y permite índices más pequeños.
- **Limitar el resultado**: paginar (`FETCH FIRST 50 ROWS ONLY` con `ORDER BY o.order_date DESC`) y filtrar por rango de fechas. Rara vez se necesitan todos los pedidos históricos de un país.
- Evitar funciones sobre columnas indexadas (`UPPER(country) = 'MÉXICO'` anula el índice), o crear un índice basado en función si es inevitable.

```
SELECT o.order_id, o.order_date, c.customer_name, o.total_amount
FROM   customers c
JOIN   orders o ON o.customer_id = c.customer_id
WHERE  c.country_code = 'MX'
AND    o.order_date >= ADD_MONTHS(TRUNC(SYSDATE), -3)
ORDER  BY o.order_date DESC
FETCH FIRST 50 ROWS ONLY;
```

6. Desnormalización y caché

- Copiar `country_code` en `orders` (mantenido por la aplicación o un *trigger*) e indexar `orders(country_code, order_date)`: elimina el join para este caso de uso. Costo: redundancia y consistencia.
- **Result Cache** de Oracle (`/*+ RESULT_CACHE */`) o caché en la aplicación (Redis) para resultados que se repiten con alta frecuencia y cambian poco.

Resumen

Estrategia	Impacto	Esfuerzo	Riesgo / costo
Estadísticas + histogramas	Medio	Muy bajo	Ninguno
Índices de cobertura / FK	**Alto**	Bajo	Escrituras más costosas, espacio
Vista materializada	Alto	Medio	Refresco, espacio
Particionamiento	Alto con filtro por fecha	Alto	Licencia EE, rediseño
Reescritura (ISO, paginación, rango)	Alto	Bajo-medio	Cambio en la aplicación
Desnormalización / caché	Alto en lecturas	Medio	Consistencia

Recomendación: 1 + 2 + paginación/rango de fechas resuelven la mayoría de los casos. La vista materializada o el particionamiento se justifican si, después de medir, la consulta sigue fuera del SLO.

Scripts

- `sql/oracle/01_diagnostico.sql` – planes y estadísticas
- `sql/oracle/02_indices.sql`
- `sql/oracle/03_vista_materializada.sql`
- `sql/oracle/04_particionamiento.sql`
- `sql/postgres/benchmark.sql` – reproduce el escenario con datos sintéticos y compara el plan antes y después de los índices (`psql -f`).

Módulo 4 – Incorporar un fix de seguridad de `main` en `feature/new-login`

Respuesta corta

```
git cherry-pick -x <sha-del-fix>
```

`git cherry-pick` aplica **únicamente** el cambio introducido por ese commit sobre la rama actual, sin traer el resto de la historia de `main`. Un `git merge main` o un `git rebase main` traerían *todas* las actualizaciones de `main`, que es justo lo que no se quiere.

`-x` agrega al mensaje la línea `(cherry picked from commit <sha>)`: queda trazabilidad de dónde salió el cambio, algo importante en un fix de seguridad (auditoría, notas de versión, verificar luego que la feature lo tiene).

Paso a paso

```
# 1. Traer el estado actual del remoto
git fetch origin

# 2. Identificar el commit exacto del fix
git log origin/main --oneline -n 20
git log origin/main --oneline --grep="security"      # o por archivo: git log origin/main -- src/auth/
git show <sha>                                        # revisar qué cambia antes de aplicarlo

# 3. Ir a la rama de trabajo, con el árbol limpio
git switch feature/new-login
git status                                            # si hay cambios sin commit: git stash

# 4. Aplicar solo ese commit
git cherry-pick -x <sha>

# 5. Si hay conflicto: resolver, marcar y continuar (o abortar)
git status
git add <archivos-resueltos>
git cherry-pick --continue                            # git cherry-pick --abort para deshacer

# 6. Verificar y publicar
./mvnw verify                                         # o la suite de tests del proyecto
git log --oneline -n 3
git push origin feature/new-login
```

Casos especiales

Situación	Comando
El fix entró a `main` como **merge commit** de un PR	`git cherry-pick -x -m 1 <sha-del-merge>` (`-m 1` toma como base el primer padre, `main`)

Situación	Comando
El fix son **varios commits consecutivos**	`git cherry-pick -x A^..B`
Quiero revisar antes de commitear	`git cherry-pick --no-commit <sha>` y luego `git commit`
El PR se integró con *squash*	Es un solo commit normal: `git cherry-pick -x <sha>`
Ya no recuerdo si lo apliqué	`git log --cherry-pick --right-only --oneline feature/new-login...origin/main` lista lo que aún falta

¿Qué pasa cuando la feature se integre a `main`?

El commit cherry-picked tiene **otro SHA** pero el **mismo patch**. Al hacer merge o rebase de la feature sobre `main`, Git detecta que el cambio ya está (mismo *patch-id*) y no genera conflicto ni duplica el código. Se puede comprobar con:

```
git log --cherry-pick --oneline main...feature/new-login
```

Alternativas descartadas

Alternativa	Por qué no
`git merge main` en la feature	Trae **todo** lo nuevo de `main`, no solo el fix
`git rebase main`	Igual que el merge: reescribe la base con todos los cambios de `main`
Copiar el cambio a mano	Propenso a errores, pierde trazabilidad y no se reconoce como el mismo patch
`git checkout main -- archivo`	Trae el archivo entero en su versión de `main`, que puede incluir otros cambios

Buenas prácticas de proceso

- Los fixes de seguridad van en **commits atómicos** (un commit = un cambio) y, idealmente, en una rama `hotfix/*` desde `main`: así son fáciles de cherry-pickear a releases y features.
- Con varias features activas, se avisa al equipo y cada dueño hace el cherry-pick en su rama.
- Si la feature está compartida, se prefiere cherry-pick (agrega un commit) frente a reescribir historia.

Demostración reproducible

`scripts/demo-cherry-pick.sh` crea un repositorio temporal con `main` y `feature/new-login`, simula el fix de seguridad junto con otros cambios en `main`, aplica el cherry-pick y verifica que la feature recibió el fix y nada más.

```
bash scripts/demo-cherry-pick.sh
```

Módulo 5 – Liderazgo técnico y gestión

Situación: 8 desarrolladores · 40 % de deuda técnica en servicios clave · 10 incidentes críticos en el último mes · entrega comprometida en 3 semanas · sin estándares y con code review irregular · 2 juniors con brechas importantes.

Lectura del problema: los 10 incidentes y la falta de estándares son síntomas de la misma causa (cambios que llegan a producción sin control suficiente). Si no se estabiliza primero, la entrega de 3 semanas se va a consumir apagando incendios. La estrategia es **estabilizar y poner un mínimo de disciplina sin detener la entrega**, negociando alcance y no calidad.

1. Cinco prioridades para las primeras 2 semanas

#	Prioridad	Acciones concretas	Resultado esperado al día 14
1	**Contener los incidentes**	Triage de los 10 incidentes; postmortems sin culpables (30 min cada uno); identificar las 3 causas raíz más repetidas y asignar dueño y fecha a cada una; guardia rotativa con runbook mínimo.	Top 3 causas con plan; ningún incidente crítico repetido por la misma causa.
2	**Acotar la entrega de 3 semanas**	Sesión con negocio y producto para definir el MVP: qué es imprescindible el día 21 y qué puede ir en una segunda entrega. Desglose en historias pequeñas, *feature flags* y riesgos visibles.	Alcance firmado, plan por semana y demo semanal acordada.
3	**Estándares mínimos no negociables**	*Branch protection* en `main`, PR con al menos 1 aprobación, CI en verde obligatorio (build + tests + análisis estático), Definition of Done escrita y plantilla de PR. Se introducen en la semana 1, no se discuten cada vez.	100 % de los cambios a `main` vía PR revisado.
4	**Mapa de la deuda técnica**	Inventario del 40 % priorizado por **riesgo × impacto** y cruzado con los incidentes: primero la deuda que causa incidentes o que bloquea la entrega. Lo demás queda en un backlog visible.	Backlog de deuda priorizado y las 5 primeras tareas en curso.
5	**Plan para los juniors**	*Pairing* diario con un senior asignado (mentor), tareas acotadas y bien especificadas, reviews como espacio de aprendizaje, objetivos a 30-60-90 días.	Cada junior entrega al menos una historia con review aprobado sin retrabajo mayor.

2. Organización del equipo para mejorar velocidad y calidad

- **Dos frentes con capacidad explícita:**
  - **Entrega (5 personas):** 3 seniors/semi-seniors + los 2 juniors en *pairing*. Foco en el MVP de 3 semanas.
  - **Estabilidad/plataforma (3 personas):** incidentes, causas raíz, deuda que genera incidentes, CI/CD y observabilidad. **Rotan** cada sprint para que todos conozcan los servicios clave y no se cree un silo.
- **Guardia rotativa** semanal (primario/secundario) con runbooks; quien está de guardia no toma historias críticas.

- **20 % de la capacidad reservada a deuda técnica** después de la entrega; durante las 3 semanas, solo la deuda que bloquea la entrega o causa incidentes.
- **Dueños por servicio** (*ownership*): cada servicio clave tiene un responsable técnico que aprueba cambios de arquitectura.
- **Ceremonias livianas:** daily de 15 min, planning semanal, demo semanal con negocio y retro quincenal. *Tech sync* semanal de 30 min para decisiones técnicas (registradas como ADR).

3. Métricas para evaluar el desempeño del área

Dimensión	Métrica	Meta inicial (90 días)
Entrega (DORA)	Frecuencia de despliegue	De semanal a varias veces por semana
	Lead time de cambios (commit → producción)	< 3 días
Estabilidad (DORA)	Tasa de fallo en cambios	< 15 %
	MTTR (tiempo de recuperación)	< 4 horas en incidentes críticos
Calidad	Incidentes críticos por mes y tendencia	De 10 a ≤ 3
	Cobertura de tests en reglas críticas · deuda en Sonar	≥ 80 % en dominio · tendencia a la baja
Flujo	Tiempo de ciclo de PR · tiempo hasta el primer review	< 24 h hasta el primer review
Predictibilidad	Comprometido vs. entregado por sprint	≥ 80 %
Personas	Satisfacción del equipo (encuesta corta mensual) · progreso de los juniors	Tendencia positiva

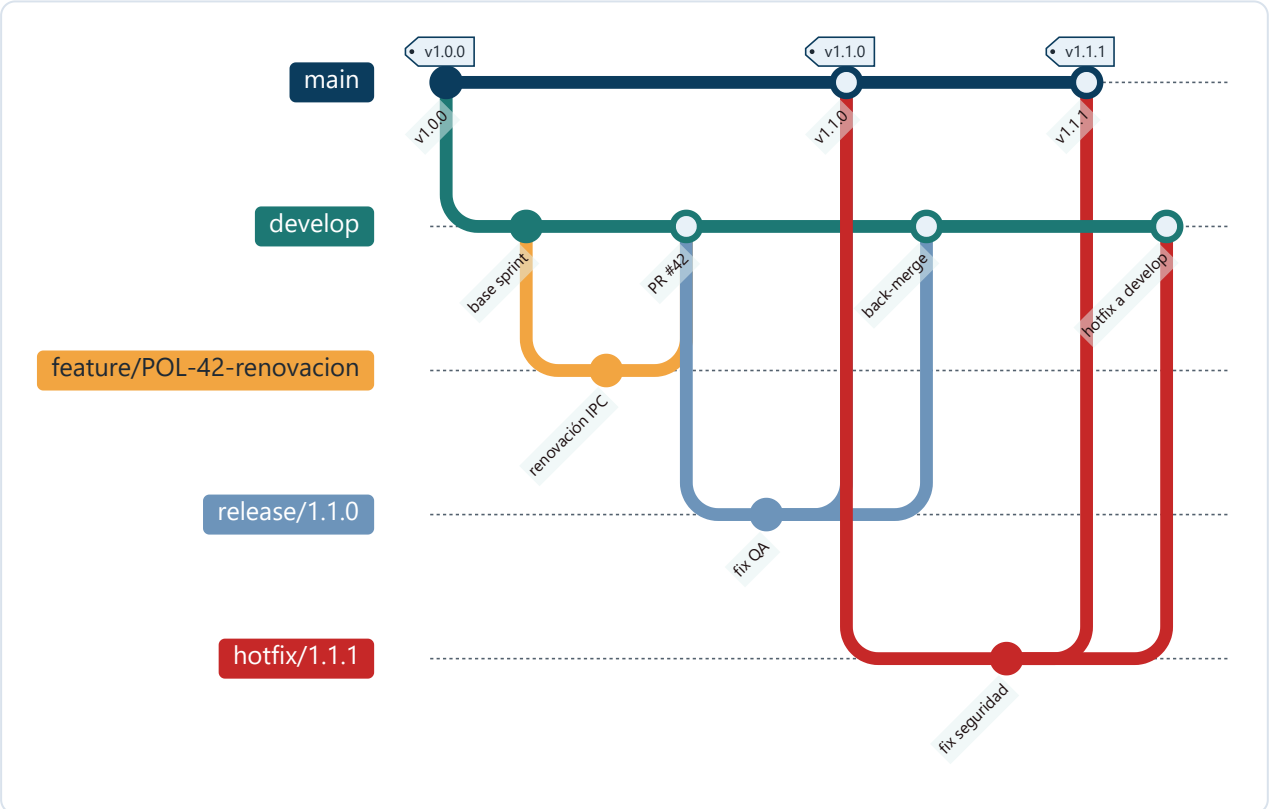
Las métricas se usan para **mejorar el sistema, no para evaluar individuos**; se revisan en la retro y se comparten con negocio de forma transparente.

4. Prácticas técnicas obligatorias

1. **Code review obligatorio:** PR con ≥ 1 aprobación (2 en servicios críticos), PR pequeños (< 400 líneas), checklist en la plantilla.
2. **CI como puerta de calidad:** build, tests, análisis estático (Sonar), escaneo de secretos y de dependencias; no se mergea en rojo.
3. **Tests en las reglas de negocio críticas** y tests de regresión para cada incidente corregido.
4. **Ramas de vida corta** (2–3 días como máximo) y *feature flags* para integrar temprano lo que aún no está listo para el usuario.
5. **Conventional Commits** y versionado semántico; changelog generado.
6. **Definition of Done compartida:** código revisado, tests, documentación mínima, observabilidad (logs, métricas, alertas) y despliegue verificado.

7. **Postmortems sin culpables** para todo incidente crítico, con acciones que tienen dueño y fecha.
8. **ADRs** para decisiones de arquitectura; IaC para toda la infraestructura.
9. **Seguridad desde el pipeline:** sin secretos en el código, dependencias actualizadas (Dependabot), mínimo privilegio.
10. **Gitflow como modelo de ramas:** encaja con un equipo que hoy no tiene estándares, entrega por versiones y convive con un CORE legado que exige releases planificados y validados.

Rama	Sale de	Se integra en	Para qué
`main`	—	—	Solo lo que está en producción; cada merge lleva un tag `vX.Y.Z`
`develop`	`main`	`release/*`	Integración continua del sprint; siempre debe compilar y pasar CI
`feature/<ticket>-<desc>`	`develop`	`develop` (PR)	Una historia por rama, vida corta, detrás de *feature flag* si no está lista
`release/X.Y.0`	`develop`	`main` + `develop`	Congela el alcance: solo correcciones, QA y notas de versión
`hotfix/X.Y.Z`	`main`	`main` + `develop` (+ `release/*` abierta)	Incidentes críticos en producción; se despliega sin esperar al sprint



Reglas que lo hacen funcionar:

- `main` y `develop` protegidas: solo se entra por PR con CI en verde y al menos una aprobación (dos en `release/*` y `hotfix/*`).

- *Squash merge* de `feature/*` hacia `develop` (historia limpia). Los merges de `release/*` y `hotfix/*` son merges normales con tag, para conservar la trazabilidad de la versión.
- **Todo `hotfix` vuelve a `develop`** (y a la `release` abierta) el mismo día. Si una feature en curso necesita el fix antes, se trae con `git cherry-pick -x` (ver Módulo 4).
- Versionado semántico con los tags de `main`; el changelog se genera a partir de los Conventional Commits.
- Ambientes: `develop` → desarrollo, `release/*` → QA/staging, `main` → producción.
- Para evitar la deuda típica de Gitflow (ramas largas, merges dolorosos): las features duran pocos días, una `release` no vive más de una semana y se revisa en cada retro si el equipo ya está listo para evolucionar a *trunk-based* cuando el CI y los *feature flags* estén maduros.

5. Gestionar la presión del negocio sin comprometer la calidad

- **Transparencia con datos:** capacidad real del equipo frente al alcance pedido, más el costo actual de los incidentes (horas perdidas, impacto en clientes). La conversación pasa de "¿pueden más rápido?" a "¿qué priorizamos?".
- **Negociar alcance, no calidad:** se define un MVP para el día 21; lo demás se entrega en incrementos. La calidad mínima (review, CI, tests en lo crítico) no está en la mesa, porque saltársela es lo que generó los 10 incidentes.
- **Entregas incrementales y visibles:** demo semanal con negocio y *feature flags* para desplegar temprano y activar cuando esté listo. Así se reduce el riesgo del "big bang" del día 21.
- **Gestión explícita de riesgos:** tablero de riesgos y escalamiento temprano (al día 7, no al día 20) si el plan se desvía.
- **Proteger al equipo:** un solo canal de entrada para pedidos urgentes (product owner), sin cambios de prioridad a mitad de semana salvo incidentes.
- **Deuda negociada como inversión:** después de la entrega, el 20 % de capacidad para deuda se presenta con su retorno esperado: menos incidentes, menor *lead time* y menor costo de cada funcionalidad nueva.

Anexo – Registros de decisiones de arquitectura (ADR)

Cada ADR documenta una decisión del Módulo 1 o del despliegue: contexto, decisión y consecuencias.

ADR 0001 – Microservicios con API Gateway

- **Estado:** aceptada
- **Fecha:** 2026-10-15

Contexto

La plataforma combina cargas con perfiles muy distintos: consultas frecuentes del back-office y de portales de inmobiliarias, una renovación por lotes que se concentra en ventanas nocturnas y notificaciones en ráfagas. Los consumidores son de dos tipos (personas autenticadas e integradores máquina a máquina) y se requiere disponibilidad 24/7.

Decisión

Separar la plataforma en servicios por capacidad de negocio (Pólizas, Riesgos, Renovación, Notificaciones, Adapter CORE) desplegados en Amazon EKS, detrás de **Amazon API Gateway**, que concentra autenticación (Cognito JWT y API keys con planes de uso), *throttling* por consumidor, validación básica y versionado por ruta (`/v1`).

Se empieza con **Pólizas y Riesgos en un mismo servicio** (comparten invariantes) y se separan solo si la carga lo justifica.

Consecuencias

- ✅ Escalado y despliegue independientes; un pico de renovaciones no degrada las consultas.
- ✅ Políticas de seguridad y cuotas en un único punto.
- ⚠️ Más componentes que operar: se mitiga con plantillas de servicio, observabilidad común y IaC.
- ⚠️ Consistencia entre servicios vía eventos (ver ADR 0002).

ADR 0002 – Event-driven con Transactional Outbox

- **Estado:** aceptada (en el código del Módulo 2 se implementa una versión simplificada)
- **Fecha:** 2026-10-15

Contexto

Cada cambio de estado de una póliza o riesgo debe (a) informarse al CORE a través del servicio de edición de WebLogic y (b) notificarse por correo/SMS en la creación y la renovación. Ninguna de las dos cosas debe bloquear ni revertir la transacción de negocio, y no se pueden perder eventos si el proceso cae justo después del commit.

Decisión

- En la misma transacción del cambio se inserta un registro en `outbox_event`.
- Un *relay* (polling o CDC con Debezium) publica en **Amazon EventBridge**; cada consumidor tiene su cola **SQS** con **DLQ**.
- Los consumidores son idempotentes (deduplicación por `eventId`).

Simplificación en el código del Módulo 2

El servicio publica un evento de dominio (`PolizaModificada`) y un `@TransactionalEventListener(AFTER_COMMIT)` asíncrono llama al CORE con retry y circuit breaker. Si el envío falla, se registra el error y la operación de negocio **no se revierte**. Lo que falta para "al menos una vez" es justamente el outbox: si el proceso cae entre el commit y el envío, el evento se pierde. Es una decisión consciente de alcance para la prueba.

Consecuencias

- ✅ La API responde sin depender de la latencia ni la disponibilidad del CORE.
- ✅ Reproceso desde la DLQ y trazabilidad por `correlationId`.
- ⚠️ Consistencia eventual con el CORE: se requiere reconciliación diaria y monitoreo de la edad de la DLQ.

ADR 0003 – Arquitectura hexagonal y capa anticorrupción para el CORE

- **Estado:** aceptada
- **Fecha:** 2026-10-15

Contexto

El CORE de seguros es un sistema legado expuesto por un servicio agnóstico de edición en WebLogic (capa media). Su contrato, sus tiempos de respuesta y sus códigos de error no deben filtrarse al dominio de pólizas, y la plataforma debe poder probarse sin él.

Decisión

- El dominio define el puerto `CorePolizasPort` (*qué* se informa: evento, póliza, riesgo, operación).
- Los adapters implementan el *cómo*: hoy `CoreHttpAdapter` (REST contra `/core-mock/evento`); mañana un adapter SOAP/JMS para WebLogic sin tocar el dominio.
- El adapter concentra resiliencia (timeout, retry con backoff exponencial y *jitter*, circuit breaker), traducción de errores y propagación del `correlationId`.

Consecuencias

- ✅ Dominio y tests independientes del CORE (en los tests de integración el puerto se reemplaza por un mock).
- ✅ Cambiar el protocolo o el proveedor es un cambio localizado.
- ⚠️ Una capa de mapeo adicional que mantener.

ADR 0004 – Despliegue demo en una EC2 con CloudFront

- **Estado:** aceptada
- **Fecha:** 2026-10-15

Contexto

La aplicación debe quedar funcional y pública durante **4 días** para la evaluación, con el menor costo posible y con toda la infraestructura definida como código (Terraform). La arquitectura objetivo (EKS, Aurora, EventBridge) cuesta del orden de cientos de dólares al mes y tarda en aprovisionarse.

Decisión

Necesidad	Solución demo	Equivalente en producción
Cómputo	1 EC2 **t4g.small** (Graviton) con Docker Compose	EKS Multi-AZ
Base de datos	PostgreSQL 16 en contenedor, volumen EBS cifrado	Aurora PostgreSQL Multi-AZ
HTTPS sin dominio	**CloudFront** con el certificado `*.cloudfront.net`	Route 53 + ACM + CloudFront
Protección del origen	SG solo con la prefix list de CloudFront + header secreto `X-Origin-Verify`	ALB interno + VPC Link
Acceso a la instancia	**SSM** (sin SSH, sin puerto 22)	SSM / bastion-less
Secretos	SSM Parameter Store (SecureString), generados por Terraform	Secrets Manager con rotación
Imágenes	**ECR** con scan on push; la instancia descarga con su rol IAM	ECR
CI/CD	GitHub Actions + **OIDC** (sin llaves estáticas)	Igual

Se eligió **ECR en lugar de GHCR**: el repositorio es privado, y con GHCR la instancia necesitaría un token de GitHub para descargar imágenes. Con ECR la autenticación es el rol IAM de la instancia, sin secretos que distribuir; el costo es de centavos.

Costo estimado para 4 días: EC2 ≈ US$1,6 + IPv4 pública ≈ US$0,5 + EBS ≈ US$0,2 + CloudFront/ECR/S3 ≈ US$0 (capa gratuita) → **menos de US$3**.

Consecuencias

- ✅ Barato, reproducible (`terraform apply`) y destruible con un workflow (`destroy.yml`).
- ⚠️ Sin alta disponibilidad: una sola instancia en una zona. Aceptable para una demo; no para producción.
- ⚠️ Postgres en contenedor: los datos sobreviven a reinicios, pero no a reemplazar la instancia.